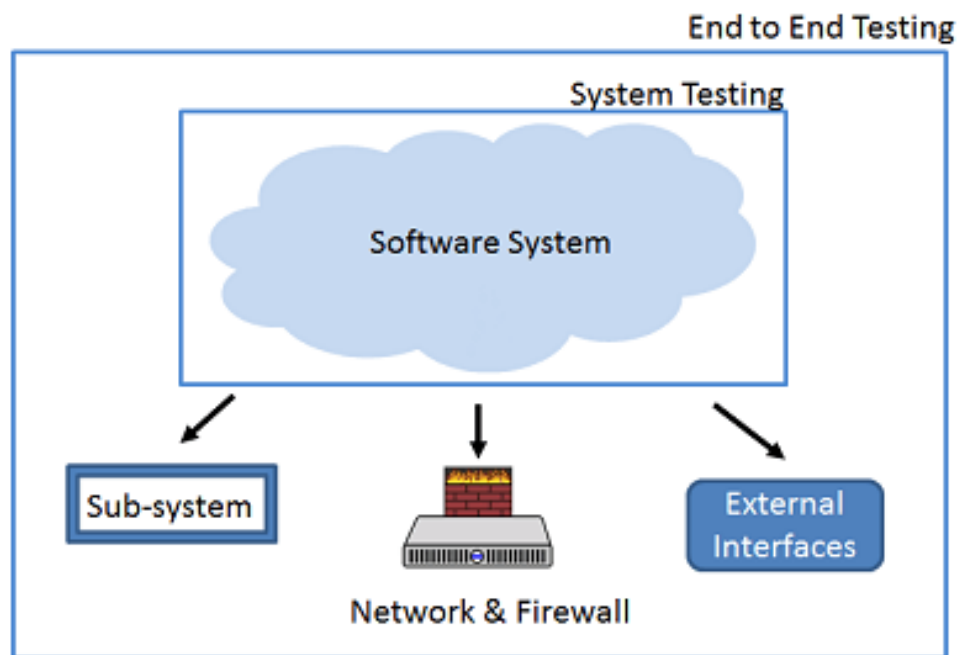


End to End Testing

What is End to End Testing?

- End-to-End Testing is a type of Software Testing that not only validates the software system under test but also checks its integration with external interfaces.
- The purpose of End-to-End Testing is to exercise a complete production-like scenario.
- Along with the software system, it also validates batch/data processing from other upstream/downstream systems.
- End to End Testing is usually executed after functional and System Testing.
- It uses actual production like data and test environment to simulate real-time settings. End-to-End testing is also called Chain Testing

What is End to End Testing?



End to End Testing vs System Testing

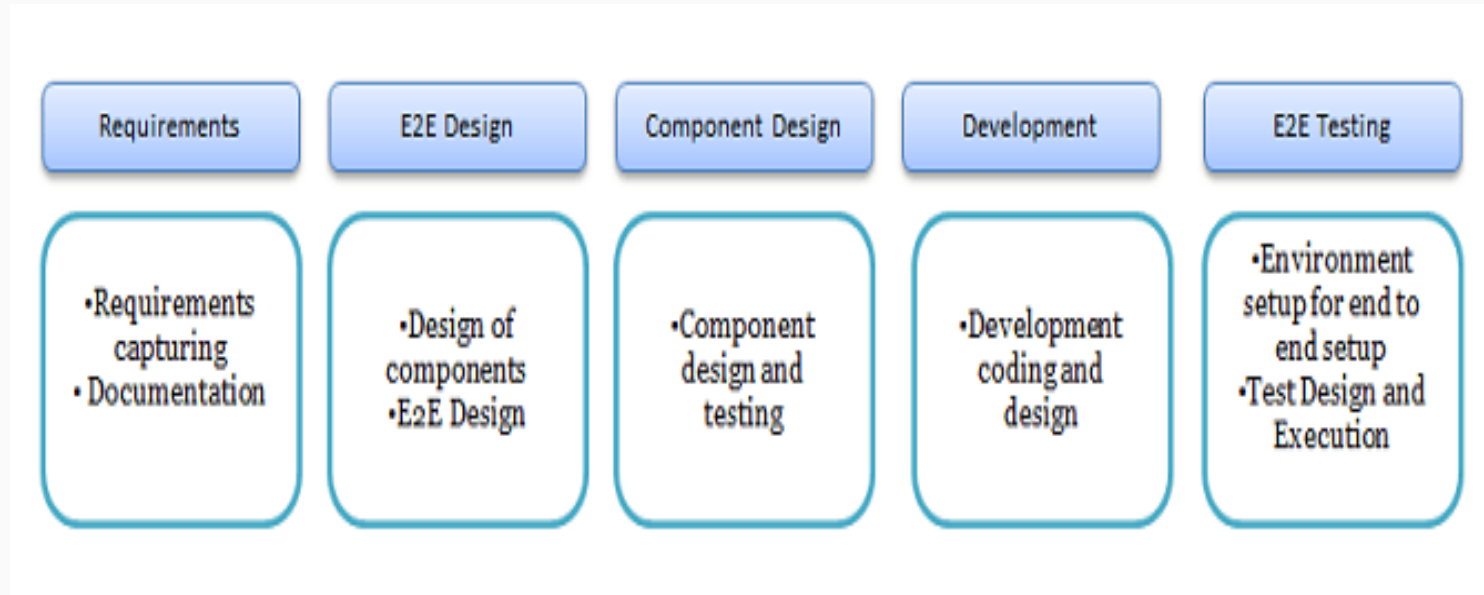
End to End Testing	System Testing
Validates the software system as well as interconnected sub-systems	Validates just the software system as per the requirements specifications.
It checks the complete end-to-end process flow.	It checks system functionalities and features.
All interfaces, backend systems will be considered for testing	Functional and Non-Functional Testing will be considered for testing
It's executed once System Testing is completed.	It's executed after Integration Testing .
End to End testing involves checking external interfaces which can be complex to automate. Hence Manual Testing is preferred.	Both Manual and Automation can be performed for System Testing

Why End to End Testing

- Modern software systems are complex and are interconnected with multiple sub-systems, which may be owned by different organizations. If any one of the sub-system fails, the whole software system could collapse. This is major risk and can be avoided by End-to-End testing.
- End-to-End testing verifies the complete system flow. It increase Test coverage of various sub-systems. It helps detect issues with sub-systems and increases confidence in the overall software product.

End to End testing Process

The following diagram gives an overview of the End to End testing process



End to End testing Process

Activities of End to End Test Include:

- Study of end to end testing requirements
- Test Environment setup and hardware/software requirements
- Describe all the systems and its subsystems processes.
- Description of roles and responsibilities for all the systems
- Testing methodology and standards
- End to end requirements tracking and designing of test cases
- Input and output data for each system

End to End testing Process

End to End Testing Design framework consists of three parts:

- Build user functions
- Build Conditions
- Build Test Cases

Build User Functions

- List down the features of the system and their interconnected components
- List the input data, action and the output data for each feature or function
- Identify the relationships between the functions
- Determine whether the functions are reusable or independent

Build Conditions based on User Function

- Building a set of conditions for each user function defined
- Conditions include sequence, timing and data conditions
- For example –login page. checking of more conditions like
 - Invalid user name and password
 - Checking with valid user name and password
 - Password strength checking
 - Checking of error messages

Building Test Cases

- For each **test scenario** , one or more test cases must be built to cover all of the functionality of the application. Test cases may include each condition as single test case.
- A Test Scenario is any functionality that can be tested. It is also called **Test Condition or Test Possibility**. As a tester, you may put yourself in the end user's shoes and figure out the real-world scenarios and use cases of the Application Under Test.

Test Scenarios—Login Page Example:

- When user browses to the app they should see the default “public” screen
- When not authenticated, user tries to access “protected” page (by browse “http://localhost:4200/protected”) they should be redirect to “login” page
- When a user is trying to login with wrong credentials, they should stay on the “login” page (without redirecting anywhere) and see an “error” message.
- When login is successful — they should be redirected to the default “public” page
- When a logged in user tries to access a protected page they should be taken to that page.